

AI 제조 기술 관련 교육

『4과목』 코딩 도구 기초 활용

2025.12.08-2025.12.19(10일 70시간)

Prepared by Daekyeong Kim

Ph.D.

Copyright 2022. Daekyeong all rights reserved

『4과목』

코딩 도구 기초 활용

- Part 1. 오리엔테이션
- Part 2. Cursor.AI 주요 기능 및 단축키
- Part 3. 토이 프로젝트 ① — Cursor 실습
- Part 4. Cursor + LangChain 통합
- Part 5. GitHub 연동 및 버전 관리
- Part 6. Prompting BP
- Part 7. 마무리 및 요약



학습목표

- 이 워크샵에서는 Cursor.AI와 LangChain을 연동하여 AI 워크플로우를 구축하고, GitHub 버전 관리까지 포함한 실무형 자동화 프로젝트를 완성하는 것에 대해 알 수 있습니다.
 - Cursor.AI의 주요 기능과 인터페이스를 이해한다.
 - 단축키를 활용하여 AI 코딩 속도를 높인다.

눈높이 체크

- Cursor.AI와 LangChain을 알고 계신가요?

절차

- “AI 워크플로우 토이 프로젝트” — 프롬프트 자동화 + 코드 생성 + 버전관리 통합 실습

모듈	주제	시간	구성
1	Cursor.AI 주요 기능 및 단축키	1.5h	이론 + 데모
2	토이 프로젝트 ① — Cursor 실습	1.5h	실습
3	Cursor + LangChain 통합	2h	이론 + 실습
4	GitHub 연동 및 버전 관리	1h	실습
5	Prompting BP	1h	회고

『4과목』

코딩 도구 기초 활용

- Part 1. 오리엔테이션
- Part 2. Cursor.AI 주요 기능 및 단축키
- Part 3. 토이 프로젝트 ① — Cursor 실습
- Part 4. 토이 프로젝트②- Cursor + LangChain 통합
- Part 5. GitHub 연동 및 버전 관리
- Part 6. Prompting BP
- Part 7. 마무리 및 요약





1. Cursor 인터페이스 구성

| Cursor 인터페이스 확인

- 파일 트리 / 코드 에디터 / AI 채팅 패널 / 인라인 프롬프트
- Command Palette: Cmd/Ctrl + K



2. 핵심 기능

| 핵심 기능 확인

- 인라인 프롬프트 (Cmd/Ctrl + I): 코드 내 직접 요청
- 리팩토링 / 테스트 생성 (Cmd/Ctrl + Shift + L)
- AI 패널: 대화형 질의응답
- Diff 미리보기 및 자동 저장 기능



3. 생산성 단축키 정리



단축키

기능	단축키
코드 자동 생성	Cmd/Ctrl + Enter
리팩토링	Alt + Shift + R
코드 설명	Cmd/Ctrl + E
대화창 열기	Cmd/Ctrl + L
프로젝트 실행	Cmd/Ctrl + B

『4과목』

코딩 도구 기초 활용

- Part 1. 오리엔테이션
- Part 2. Cursor.AI 주요 기능 및 단축키
- **Part 3. 토이 프로젝트 ① — Cursor 실습**
- Part 4. 토이 프로젝트②- Cursor + LangChain 통합
- Part 5. GitHub 연동 및 버전 관리
- Part 6. Prompting BP
- Part 7. 마무리 및 요약





1. 토이 프로젝트 ① — Cursor 실습

학습 목표

- Cursor에서 AI 코딩 실습 환경을 완성하고, 자동 코드 생성을 체험한다.



2. 실습 단계

| 프로젝트 시작

- 새 폴더 toy-project 생성
- main.py 파일 작성 후 입력:
 - 사용자 입력을 받아 단어 빈도를 세는 CLI 프로그램을 만들어줘.

2. 실습 단계

| 코드 작성

- 주어진 요청은 Python과 같은 프로그래밍 언어를 사용하여 CLI(Command Line Interface) 프로그램을 작성하는 것입니다. 이 프로그램은 사용자가 입력한 문자열에서 각 단어가 몇 번 등장했는지 빈도를 계산하는 기능을 구현해야 합니다.
- 다음은 요청에 맞는 Python 코드입니다:

```
from collections import Counter

def main():
    # 사용자 입력 받기
    user_input = input("문장을 입력하세요: ")

    # 입력된 문장을 공백 기준으로 분리하여 단어 리스트 생성
    words = user_input.split()

    # 단어 빈도 계산
    word_count = Counter(words)

    # 결과 출력
    print("\n단어 빈도:")
    for word, count in word_count.items():
        print(f'{word}: {count}번')

if __name__ == "__main__":
    main()
```



2. 실습 단계

코드 작성

1. `from collections import Counter:`

1. Counter는 Python의 collections 모듈에 포함된 클래스입니다. 리스트와 같은 반복 가능한 객체에서 각 요소의 빈도를 손쉽게 계산할 수 있습니다.

2. `user_input = input("문장을 입력하세요: ")`:

1. 사용자로부터 문자열을 입력받습니다.

3. `words = user_input.split()`:

1. 입력된 문자열을 공백() 기준으로 나누어 단어들의 리스트를 생성합니다.

4. `word_count = Counter(words)`:

1. Counter를 사용하여 각 단어의 빈도를 계산합니다. word_count는 딕셔너리와 유사한 형태로 저장됩니다. 키는 단어, 값은 해당 단어의 빈도를 나타냅니다.

5. 결과 출력:

1. for 루프를 사용하여 word_count의 각 키-값 쌍(단어와 빈도)을 출력합니다.

6. `if __name__ == "__main__":`:

1. 이 코드는 해당 스크립트가 직접 실행될 때만 main() 함수를 실행하도록 보장합니다.



2. 실습 단계

| 테스트

- 테스트 코드 생성 요청:
 - 위 코드에 대한 pytest 테스트 코드를 만들어줘.



2. 실습 단계

테스트

- 아래는 예제로 사용할 간단한 Python 함수와 해당 함수에 대한 pytest 테스트 코드입니다.

- 예제 코드

```
def add(a, b):  
    """두 숫자의 합을 반환합니다."""  
    return a + b
```

```
def subtract(a, b):  
    """두 숫자의 차를 반환합니다."""  
    return a - b
```

- 위 코드는 두 개의 숫자를 더하거나 빼는 간단한 함수입니다.

2. 실습 단계

테스트

- pytest 테스트 코드 작성
- 위 코드를 테스트하기 위해 pytest를 사용한 테스트 코드는 다음과 같습니다:

```
import pytest
from your_module_name import add, subtract # 실제 파일 이름으로 변경하세요.
```

```
def test_add():
    assert add(3, 5) == 8 # 3 + 5 = 8
    assert add(-1, 1) == 0 # -1 + 1 = 0
    assert add(0, 0) == 0 # 0 + 0 = 0

def test_subtract():
    assert subtract(10, 5) == 5 # 10 - 5 = 5
    assert subtract(0, 3) == -3 # 0 - 3 = -3
    assert subtract(-1, -1) == 0 # -1 - (-1) = 0
```

● 실행 방법

- 위의 테스트 코드를 test_your_module_name.py와 같은 파일로 저장합니다.
- 터미널에서 pytest 명령어를 실행합니다:
- `pytest test_your_module_name.py`
- 모든 테스트가 성공하면 출력에 PASSED 메시지가 나타납니다.



2. 실습 단계

리팩토링

- 리팩토링 요청:
 - 함수를 분리하고 예외 처리 코드를 추가해줘.



2. 실습 단계

리팩토링

- 이 문장은 기존 코드에서:
 - 기능을 별도의 함수로 분리하고,
 - 예외 상황을 처리하기 위한 코드를 추가하라는 작업 지시입니다.
- 예를 들어 Python에서 아래와 같은 방식으로 구현할 수 있습니다:

기존 코드

```
result = int(input("숫자를 입력하세요: ")) / int(input("나눌 숫자를 입력하세요: "))  
print(result)
```

2. 실습 단계

| 추가 기능

● # 요청에 따라 수정된 코드

```
def divide_numbers():  
    try:  
        num1 = int(input("숫자를 입력하세요: "))  
        num2 = int(input("나눌 숫자를 입력하세요: "))  
        result = num1 / num2  
        return result  
    except ZeroDivisionError:  
        print("0으로 나눌 수 없습니다.")  
    except ValueError:  
        print("올바른 숫자를 입력하세요.")  
    except Exception as e:  
        print(f"예기치 못한 오류가 발생했습니다: {e}")
```

```
# 함수 호출  
print(divide_numbers())
```

● 위 수정된 코드에서:

- divide_numbers 함수로 기능을 분리하였고,
- 다양한 예외 상황(예: ZeroDivisionError, ValueError)을 처리하는 코드를 추가하였습니다.



2. 실습 단계

리뷰

- 실행 및 결과 확인
- Cursor가 함수 구조를 제안하고 자동으로 테스트 케이스 생성함.

『4과목』

코딩 도구 기초 활용

- Part 1. 오리엔테이션
- Part 2. Cursor.AI 주요 기능 및 단축키
- Part 3. 토이 프로젝트 ① — Cursor 실습
- **Part 4. 토이 프로젝트②- Cursor + LangChain 통합**
- Part 5. GitHub 연동 및 버전 관리
- Part 6. Prompting BP
- Part 7. 마무리 및 요약





1. 학습 목표

| RAG 실습

- LangChain의 기본 구조를 이해하고, Cursor 내에서 간단한 AI 파이프라인을 구축한다.

2. LangChain 기본 구성

| RAG 실습

- LLM → PromptTemplate → LLMChain → OutputParser
- Memory / Retrieval / Tools 구조

3. Cursor와의 관계

| RAG 실습

- Cursor = 개발환경
- LangChain = 프롬프트 실행·조합 엔진



4. 활용 예시

| RAG 실습

- 문서 요약, 질문응답, 데이터 분석 자동화

5. 실습

| 미니 RAG형 챗봇 만들기

- app.py 파일 작성
- 프롬프트 입력:

OpenAI API를 이용한 LangChain 스크립트를 작성해줘.

- - 텍스트 파일을 로드하고
- - 사용자 질문에 맞춰 요약 답변을 생성하고
- - 질문과 답변을 함께 출력하도록 해줘.

미니 RAG형 챗봇 만들기

- 주어진 요청은 OpenAI API를 활용하여 LangChain을 사용한 스크립트를 작성하는 것입니다. 이 스크립트는 다음과 같은 작업을 수행합니다:
- 텍스트 파일 로드: 지정된 텍스트 파일을 읽어 내용을 가져옵니다.
- 질문에 맞춘 요약 답변 생성: 사용자가 입력한 질문에 대해 텍스트 파일의 내용을 바탕으로 OpenAI API를 사용하여 요약된 답변을 생성합니다.
- 질문과 답변 출력: 사용자가 한 질문과 이에 대한 답변을 함께 출력합니다.
- 아래는 해당 요청을 구현한 Python 코드 예제입니다.

5. 실습

미니 RAG형 챗봇 만들기

● 코드 예제:

```
from langchain.llms import OpenAI
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain
```

```
# 텍스트 파일 로드 함수
```

```
def load_text_file(file_path):
    with open(file_path, 'r', encoding='utf-8') as file:
        return file.read()
```

```
# LangChain 기반 스크립트
```

```
def main():
```

```
    # 1. OpenAI API 모델 초기화
```

```
    llm = OpenAI(temperature=0.7, model="text-davinci-003") # 적절한 모델 선택
```

```
    # 2. 텍스트 파일 로드
```

```
    file_path = "example.txt" # 분석할 텍스트 파일 경로
```

```
    text_data = load_text_file(file_path)
```

```
    # 3. 사용자 질문 입력받기
```

```
    user_question = input("질문을 입력하세요: ")
```

```
    # 4. 프롬프트 템플릿 정의
```

```
    prompt_template = PromptTemplate(
```

```
        input_variables=["text", "question"],
```

```
        template="다음 텍스트를 바탕으로 사용자의 질문에 답변하세요:\n\n텍스트:\n{text}\n\n질문:\n{question}\n\n
```

```
답변:"
```

```
)
```

5. 실습

미니 RAG형 챗봇 만들기

5. LLMChain 생성

```
chain = LLMChain(llm=llm, prompt=prompt_template)
```

6. 질문에 대한 답변 생성

```
answer = chain.run({"text": text_data, "question": user_question})
```

7. 질문과 답변 출력

```
print("\n[질문]")
print(user_question)
print("\n[답변]")
print(answer)
```

실행

```
if __name__ == "__main__":
    main()
```

load_text_file 함수:

지정된 경로의 텍스트 파일을 읽고 내용을 문자열로 반환합니다.

LangChain 구성:

OpenAI 객체를 생성하여 OpenAI의 언어 모델(예: text-davinci-003)을 초기화합니다.

PromptTemplate를 통해 질문에 대한 답변을 생성하기 위한 프롬프트 형식을 정의합니다.

LLMChain은 모델과 프롬프트를 결합하여 입력 데이터를 처리합니다.

사용자 입력 처리:

사용자로부터 질문을 입력받아 이를 프롬프트와 함께 전달합니다.

텍스트 파일의 내용(text_data)과 사용자 질문(user_question)을 프롬프트에 삽입하여 답변을 생성합니다.

출력:

사용자 질문과 생성된 답변을 출력합니다.

실행 방법:

텍스트 파일(example.txt)을 준비합니다.

코드를 실행한 후, 질문을 입력하면 해당 텍스트를 기반으로 요약된 답변이 출력됩니다.

5. 실습

| 미니 RAG형 챗봇 만들기

- Cursor에서 코드 자동 생성 → 실행
- 프롬프트 확장:
 - 최근 3개의 대화를 기억하도록 ConversationBufferMemory를 추가해 줘.
- 응답 결과를 ChatGPT 결과와 비교
- LangChain + Cursor 조합을 통해 프롬프트 체인 자동화 구현.

미니 RAG형 챗봇 만들기

- 주어진 문장은 프로그래밍이나 인공지능 시스템에서 사용되는 개념을 설명하는 문장으로 보입니다. 이를 설명하자면:
- "최근 3개의 대화를 기억하도록 ConversationBufferMemory를 추가해줘."
- 이 문장은 대화형 AI 시스템에서 사용되는 메모리 관리와 관련된 기능을 설정하는 요청입니다. 구체적으로, ConversationBufferMemory라는 메모리 관리 모듈 또는 클래스(혹은 기능)를 사용하여, AI가 최근에 이루어진 3개의 대화만 기억하도록 설정하겠다는 의미입니다.
- 주요 내용:
 - ConversationBufferMemory:
 - 대화형 AI에서 대화 기록을 관리하는 메모리 기능입니다.
 - AI가 이전 대화를 기억하고, 이를 기반으로 더 자연스럽게 연속적인 대화를 이어나갈 수 있도록 돕습니다.
 - 최근 3개의 대화만 기억:
 - 메모리에 저장할 대화의 개수를 제한하여, 시스템이 이전 대화 중 최근 3개만 기억하도록 설정합니다.
 - 이는 메모리 사용량을 줄이고, 불필요한 과거 데이터를 제거하여 효율성을 높이는 데 도움을 줍니다.

『4과목』

코딩 도구 기초 활용

- Part 1. 오리엔테이션
- Part 2. Cursor.AI 주요 기능 및 단축키
- Part 3. 토이 프로젝트 ① — Cursor 실습
- Part 4. 토이 프로젝트②- Cursor + LangChain 통합
- Part 5. GitHub 연동 및 버전 관리
- Part 6. Prompting BP
- Part 7. 마무리 및 요약





1. 학습 목표

| GitHub 이용

- GitHub를 이용해 Cursor 프로젝트의 버전을 관리하고 협업 기반으로 운영한다.



2. Git 기본 명령

| GitHub 이용

- `git init`
- `git add .`
- `git commit -m "init"`
- `git remote add origin ...`
- `git push -u origin main`

3. Cursor 내 Git 연동

| GitHub 이용

- Diff 비교 / Commit / Branch 생성 가능

4. 버전 관리 모범 사례

| GitHub 이용

- 명확한 커밋 메시지(feat:, fix:, refactor:)
- .gitignore로 환경파일 제외
- 주요 변경 전 항상 commit/push



5. 실습

| GitHub 이용

- GitHub에서 새 저장소 생성
- Cursor 프로젝트 연결 (git remote add origin)
- 커밋 & 푸시
- 새 브랜치 생성 feature/langchain
- 병합 후 결과 확인

『4과목』

코딩 도구 기초 활용

- Part 1. 오리엔테이션
- Part 2. Cursor.AI 주요 기능 및 단축키
- Part 3. 토이 프로젝트 ① — Cursor 실습
- Part 4. Cursor + LangChain 통합
- Part 5. GitHub 연동 및 버전 관리
- Part 6. Prompting BP
- Part 7. 마무리 및 요약





1. Sprint Retrospective

Unit

1

개요

- ❖ 지난 스프린트에서 잘한 점과 잘못된 점을 공유
- ❖ 회고 미팅은 각 이터레이션 말에 한다. 팀에서 잘 진행된 점, 잘 진행되지 못한 점, 그리고 팀의 개선을 위한 아이디어에 대해 생각해볼 기회이다.
 - ✓ 가치 있는 소프트웨어 전달
 - ✓ 지속 가능하게 작업
 - ✓ 팀원 서로와 사용자와 의사소통 잘 하기
 - ✓ 기술 향상시키기
 - ✓ 효과적으로 일하기
 - ✓ 개선전략 찾기
- ❖ 진행
 - ✓ 30~60분 정도
 - ✓ 모든 스프린트에서 수행



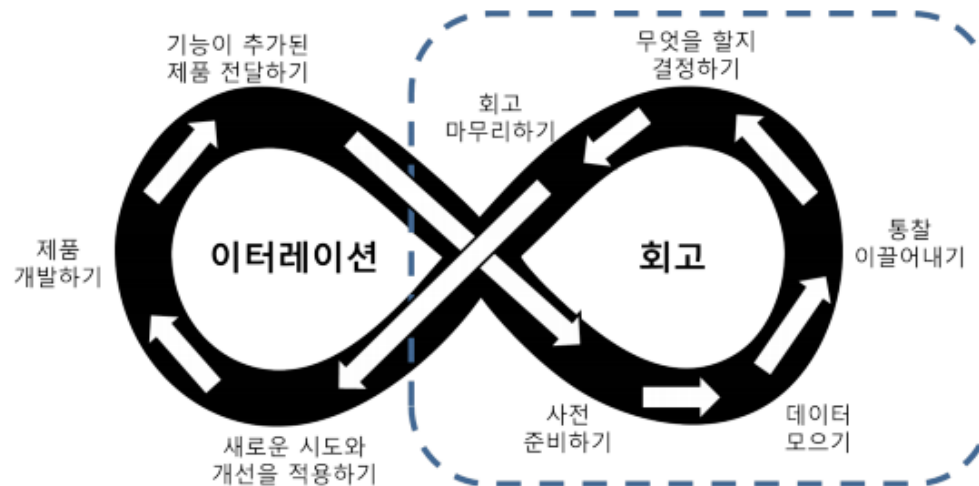
1. Sprint Retrospective

Unit

1

절차

- ❖ 애자일 회고는 이터레이션 동안 팀이 일해온 방식과 결과물에 대해 팀이 함께 이야기 하면서 개선할 점을 찾고, 다음 이터레이션에 보다 나은 팀이 되기 위해 개선 사항을 적용 하는 실천법입니다.





1. Sprint Retrospective

Unit

1

목적

- ❖ 개발 프로세스에서 개선될 사항을 식별할 기회 갖기
- ❖ 매 이터레이션이 종료될 때 가지는 미팅으로서 개발 프로세스(및 기타 지원 프로세스)에 대한 잠재적 개선 사항을 식별하고 합의한다.
- ❖ 회고는 팀과 제품 책임자가 완료된 이터레이션을 돌아보면서 프랙티스에 대한 개선 사항을 식별할 수 있는 기회이다.
- ❖ 일정기간 동안 프로젝트를 수행한 뒤 팀이 프로세스를 개선하고 추후 동일한 문제점을 반복하지 않기 위해 실시합니다.
 - ✓ 온고이지신(溫故而知新)의 개념으로 과거를 돌아보며 새로운 것을 깨닫는 활동입니다.
 - ✓ 팀이 스스로 학습하고 개선해 나갈 수 있도록 하는 장치로, 애자일 실천법 중 가장 먼저 도입할 것을 추천합니다.
 - ✓ 팀의 단점을 찾기보다, 강점을 찾아 극대화 하는 것을 목표로 합니다.



1. Sprint Retrospective

Unit

1

회고 시기

- ❖ 회고는 이터레이션의 종료 시에 수행되는 제한된 시간의 활동이다.
- ❖ 릴리즈 후에 회고를 수행해 릴리즈 계획의 성공 여부를 살펴보면서 릴리즈 내의 모든 이터레이션에 대해 돌아보는 것도 가능하다



1. Sprint Retrospective

Unit

1

회고역할

- ❖ 회고의 참여자는 다음과 같다.
- ❖ 팀원, 전문가, 팀 리더 또는 (사용자를 대표하는) 제품 책임자.
 - ✓ 잘된 것과 보다 잘될 수 있는 것에 대한 의견 제시
 - ✓ 작업 프랙티스의 개선을 위한 아이디어 제시
 - ✓ 다음 이터레이션에 이루어질 할 개선 사항에 대한 합의
- ❖ 팀 리더
 - ✓ 미팅의 진행을 돕고 모든 사람에게 기회를 부여하고 사람들이 핵심에서 벗어나지 않게 하며 제한된 시간 내에 미팅 끝내기
 - ✓ 합의된 개선 사항이 구현되도록 하기
- ❖ 역할자
 - ✓ 퍼실리테이터: 회고의 진행이 매끄럽게 수행되도록 참가자들의 관심과 참여를 유도합니다.
 - ✓ 참가자: 함께 프로젝트를 수행한 팀원들이 참여합니다.
- ❖ 적응상황
 - ✓ 팀이 같은 실수를 반복한다고 느껴질 때
 - ✓ 조직이 비슷한 상황을 반복해서 겪게 될 때
 - ✓ 팀의 불만이나 제안이 리더에게 전달되지 않을 때

1. Sprint Retrospective

Unit

1

실천법





1. Sprint Retrospective

Unit

1

실천법

Analysis		Actions		
무엇이 잘 되었습니까?	무엇을 개선 할 수 있습니까?	무엇을 그만 둘까요?	무엇을 계속해야합니까?	무엇을 시작해야합니까?
더 체계적	더 빠른 결정	재 작업	Focus	스토리 업데이트에 의한 스탠드 업
짧은 회의	재 작업	너무 많은 회의	체계적	회의 전 설명
더 빠른 결정	4/30 전후의 선명도		짧은 회의	오전 10:05에 스탠드 업
Board	회의 전 토론		더 빠른 결정	4/30 전후의 선명도
스탠드 업	너무 많은 회의			
결과 중심 회의	시간 기대			
	이메일 인플레이션			
	정시에 시작			



1. Sprint Retrospective

Unit

1

산출물

- ❖ 회고 미팅의 구체적인 작업 산출물은 없다. 개선을 위한 아이디어는 팀이 다음 이터레이션에서 구현하거나 제품 백로그의 항목이 되게 하기 위한 충분한 노력을 요할 수 있다. 일부 개선 사항(예: 외부로부터 지원되는 도구의 변경)은 팀 리더가 프로젝트 외부의 이해 관계자와 접촉할 것을 요구한다



1. Sprint Retrospective

Unit

2

순서

- ❖ 사전 준비하기:
 - ✓ 회고에 참여할 사람들을 소집하고 회고할 수 있는 분위기를 만듭니다. 경직되지 않은 열린 마음을 가질 수 있도록 합니다.
- ❖ 자료 모으기:
 - ✓ 기간 동안에 벌어졌던 다양한 자료들을 수집합니다. 시간이나 사건에 따라 모을 수 있습니다. 잘 했던 점, 아쉬웠던 점들을 나열합니다. 잘 했던 점을 더 잘하게, 혹은 미진했던 점을 극복할 수 있는 방법이 무엇인지를 모읍니다.
- ❖ 통찰 이끌어내기:
 - ✓ 이렇게 모여진 아이디어들을 바탕으로 투표를 하여 실제 다음 기간 동안 실행에 옮길 아이템을 선정합니다.
- ❖ 무엇을 할지 결정하기:
 - ✓ 구체적이고, 측정할 수 있고, 달성 가능하며, 적절하고, 시기적절한(S.M.A.R.T) 목표를 세우도록 합니다.
- ❖ 회고 끝내기:
 - ✓ 회고에 참석한 사람들에게 감사를 표현합니다. 회고 자체를 더 잘하기 위한 방법을 나눕니다.



1. Sprint Retrospective

Unit

2

유용한 단계 별 회고 기법들

단계	기법 예제
체크인/회고 준비하기	① 이터레이션에 대한 감상 공유 ② ESVP ③ 체크인
데이터 모으기	① 좋았던 점 / 나빴던 점 모으기 ② 학습 매트릭스 ③ 화남, 기쁨, 슬픔 ④ 팀 만족도 그래프 그리기
통찰 이끌어내기	① 점 투표 하기
무엇을 할 지 결정하기	① SMART 한 회의 ② EXACT 한 목표기준 세우기
회고 마무리하기	① 회고활동에 대한 회고 ② 서로에게 감사하기 ③ Follower 구하기

2. Retrospective 절차



체크인/회고 준비하기

체크인

- ❖ Time Box: 5분
- ❖ 수행 방법
 - ✓ 회의시작 전에 구성원들에게 특정 주제에 대해 짧게 돌아가며 이야기 합니다.
- ❖ 주제 예시
 - ✓ 지금 자신이 느끼고 있는 감정을 한 마디로 표현해 주세요
 - ✓ 지금 자신에게 가장 필요한 것을 한 마디로 표현해 주세요.
 - ✓ 최근 있었던 에피소드와 그로 인해 생긴 변화는 무엇이 있나요.
 - ✓ 요즘 자신이 가장 좋아하는 음식은 무엇인가요
 - ✓ 자신을 동물에 비유한다면 어떤 동물일까요.
- ❖ 주의할 점
 - ✓ 심리적인 장벽을 낮춰줄 뿐이지, 궁극적으로 모임이나 회의의 문제점을 해결해 주지 않습니다.
- ❖ 기대효과
 - ✓ 적극적으로 참여하지 않았던 사람에게 무조건 참여하는 기회를 줌으로써, 참여에 대한 두려움과 심리적 장벽을 없애 보다 회의에 적극적으로 참여하도록 합니다.
 - ✓ 모임이나 회의에서 보다 다양한 의견들이 나옵니다.

2. Retrospective 절차



체크인/회고 준비하기

이터레이션에 대한 감상 공유

- ❖ 사람들이 손쉽게 회고에 집중하기 위해, 분위기를 조성하는 활동으로, 지난 이터레이션 결과물(완료항목과 테스트 결과)에 대한 공유를 수행합니다.
- ❖ Time Box: 10분
- ❖ 수행 방법
 - ✓ 팀원 한 명씩 번갈아 가며, 이번 이터레이션에서 느꼈던 것을 짧게 이야기 하도록 합니다.
 - ✓ 내용은 자유롭게 말할 수 있도록 하되, 시간은 개인별로 1분 30초 이내로 제한합니다.
- ❖ 주의할 점
 - ✓ 내용 자체가 너무 부정적으로 흐르지 않도록 합니다.
 - ✓ 팀 내 인원에 대한 불만을 이야기 하지 않도록 합니다.
- ❖ 기대효과
 - ✓ 이터레이션을 수행하면서 느꼈던 감정과 데이터들을 정리할 수 있는 시간을 제공합니다.
 - ✓ 나 이외에 다른 팀원들이 지난 이터레이션 동안에 생각나는 내용을 공유합니다.



2. Retrospective 절차



체크인/회고 준비하기

ESVP

❖ Time Box: 10분

❖ 수행 방법

- ✓ 팀원 한 명 당 한 개의 포스트잇을 나누어 줍니다.
- ✓ 참가자들은 익명으로 회고에 대한 자신의 태도를 ESVP 중 알파벳 하나로 적게 합니다.
 - E: Explorer (탐험가)
 - 새로운 아이디어나 직관을 쉽게 발견하는 사람. 가능한 모든 것을 배우고 싶어하는 사람
 - S: Shopper (쇼핑하는 사람)
 - 가능한 한 모든 정보를 둘러보고 그 중 유용한 아이디어를 취하고 행복해하는 사람
 - V: Vacationer (휴양객)
 - 현재 회고에 흥미는 없지만, 매일 반복되어 지겨운 일상에서 잠시나마 떠나있는 행복을 느끼는 사람
 - P: Prisoner (죄수)
 - 억지로 앉아 있다고 느끼고, 차라리 다른 일을 하고 싶어하는 사람
- ✓ 결과를 칠판에 붙여, ESVP 가 각각 몇 명인지 팀원 전체가 알게 합니다.
- ✓ “이 결과에 대해 어떻게 생각하나요?” 라는 질문으로 보다 심도 있는 대화로 진행하는 기초로 활용합니다.

2. Retrospective 절차



체크인/회고 준비하기

❖ 기대효과

- ✓ 이터레이션을 수행하면서 느꼈던 감정과 데이터들을 정리할 수 있는 시간을 제공합니다.
- ✓ 나 이외에 다른 팀원들이 지난 이터레이션에 생각난 내용을 공유합니다.



2. Retrospective 절차



데이터 모으기

좋았던 점 / 나빴던 점 모으기

- ❖ 지난 이터레이션 동안 일어난 일에 대해 팀원이 느낀 공통 분모를 찾는 활동
- ❖ Time Box: 20분
- ❖ 수행 방법
 - ✓ 참여 인원에게 두 가지 색깔의 포스트잇을 각각 3 장씩 나눠줍니다.
 - ✓ 참여 인원에게 이터레이션 동안 일하는데 도움이 되었던 요소와 방해가 되었던 요소를 각각 다른 색의 포스트잇에 적게 합니다.
 - ✓ 작성이 완료된 사람의 포스트잇부터, 왼쪽에는 도움이 된 요소, 오른쪽에는 방해가 되었던 요소로 벽에 붙이고 비슷한 내용끼리 그룹핑합니다.
 - ✓ 진행자는 그룹핑된 내용을 간략하게 정리하여 전체 인원과 공유합니다.
- ❖ 수행 팁
 - ✓ 인원이 많을 경우(10 명 이상) 시간을 단축시키기 위해, 포스트잇을 3 장 대신 2 장만 사용합니다.
 - ✓ 포스트잇이 많을 경우 벽에 붙이는 시간을 줄이기 위해, 팀원 중 한 명에게 도움을 요청합니다.
- ❖ 기대효과
 - ✓ 아주 가벼운 내용부터 무거운 내용까지 다양한 의견이 공유될 수 있는 장을 만듭니다.
 - ✓ 비슷한 내용이 많다면 꼭 개선해야 할 내용일 수 있습니다.

2. Retrospective 절차



데이터 모으기

학습 매트릭스

❖ Time Box: 30분

❖ 수행 방법

- ✓ 화이트 보드에 X/Y 축을 그리고, 왼쪽 위부터 잘된 점(+), 나뻐던 점(-), 고칠 내용(C), 불확정/기타(?) 을 적습니다.
- ✓ 진행자는 해당 내용에 대해 팀원들에게 설명하고 20 분 동안 자유롭게 화이트 보드에 와서 이터레이션에 대한 아이디어를 적도록 합니다.

❖ 수행 팁

- ✓ 고칠 내용이 다음 이터레이션에 적용가능한지 팀에 스스로에게 물어보게 합니다.

❖ 기대효과

- ✓ 팀 스스로 이슈 및 현안에 대해 자유롭게 공유할 수 있게 만들어
- ※ KPT 회고 (Keep, Problem, Try)를 활용할 수 있습니다.
Self-Organizing을 연습할 수 있게 해줍니다.

+	-
C	?

2. Retrospective 절차



데이터 모으기

화남, 기쁨, 슬픔

❖ Time Box: 20분

❖ 수행 방법

- ✓ 참여자에게 각각 다른 색깔(빨간색, 노란색, 파란색)의 포스트잇 3장을 나눠줍니다.
- ✓ 이터레이션 동안 기억날 만한 이벤트들을 상기시켜 줍니다.
- ✓ 이터레이션 동안 가장 화가 났던 이벤트를 빨간색 포스트잇에 적고, 가장 기뻐했던 이벤트를 노란색 포스트잇에 적고, 가장 슬펐던 이벤트를 파란색 포스트잇에 적게 합니다.
- ✓ 진행자는 포스트잇을 수집하여 3 그룹으로 나눠 왼쪽, 가운데, 오른쪽에 각각 화남, 기쁨, 슬픔을 적습니다.
- ✓ 진행자는 해당 내용을 리포팅하면서 각각 이벤트들에 대해 상기시킵니다.

2. Retrospective 절차



데이터 모으기

화남, 기쁨, 슬픔

❖ 수행 팁

- ✓ 이 회고 방식은 부정적인 의견을 보다 많이 도출시키기 때문에, 긍정적인 내용을 나중에 이야기 해야 좋은 분위기로 나머지 회고를 진행할 수 있습니다. 따라서 슬픔→화남→기쁨 순서대로 리포팅하는 것이 좋습니다.

❖ 기대효과

- ✓ 부정적인 감정을 긍정적인 감정과 적절히 균형을 유지하여, 문제를 팀이 직시하고 해결할 수 있는 장을 만듭니다.



2. Retrospective 절차



데이터 모으기

팀 만족도 그래프 그리기

❖ Time Box: 20분

❖ 수행 방법

- ✓ 참여 인원들에게 포스트잇 한 장씩을 나누어 줍니다.
- ✓ A4 지에 다음이 적혀있는 종이를 나누어 주고, 포스트잇에 개인이 생각하는 팀의 수준을 숫자로 적게 합니다.
- ✓ 각 레벨에 대한 정의를 모두에게 공유합니다.
 - Lv 5: 우리 팀은 세계 최고의 팀이다.
 - Lv 4: 나는 내가 이 팀의 일원이어서 기쁘고 우리 팀이 협력하는 방식에 만족한다.
 - Lv 3: 제법 만족한다. 우리는 협업을 잘하는 편이다.
 - Lv 2: 만족할 때도 있지만 그렇지 않을 때도 있다.
 - Lv 1: 불행하다. 팀 워크 수준이 불만족스럽다.



2. Retrospective 절차



데이터 모으기

팀 만족도 그래프 그리기

- ✓ 벽에 별 수치를 알기 쉽게 숫자 별 일렬로 포스트잇을 나열하여 붙입니다.
- ✓ 팀원들에게 5분 정도의 시간을 주고, 우리 팀 전체의 만족도가 현재 상태인 이유를 고민하게 합니다.
- ✓ 팀원 한 명씩 차례로 우리 팀의 상태에 대한 원인을 추측한 내용을 발표하게 합니다.

❖ 수행 팁

- ✓ 개선할 내용이 다음 이터레이션에 적용 가능한지 팀 스스로 정해야 합니다.

❖ 기대효과

- ✓ 팀 스스로 이슈 및 현안에 대해 자유롭게 공유할 수 있게 만들어 Self-Organizing을 연습할 수 있게 해줍니다.

팀워크 만족도





2. Retrospective 절차



통찰 이끌어내기

점 투표하기

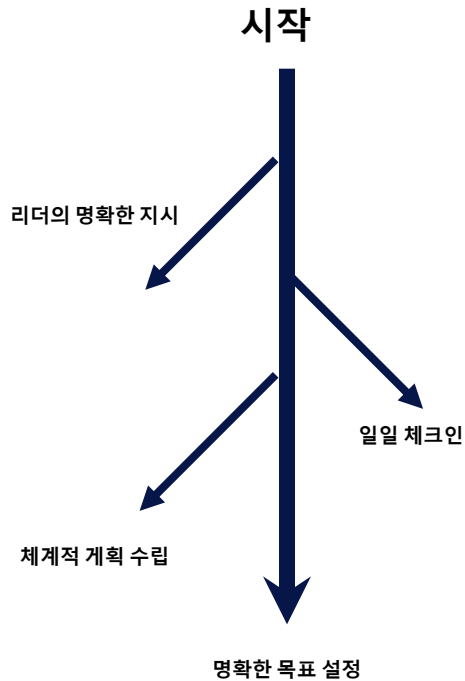
- ❖ 모아진 데이터를 이용하여 어떠한 주제에 대해 토의할 지 논의하여, 보다 개선된 팀이 되기 위해 무엇을 해야 할 지에 관한 실천계획(Action Plan)을 만드는 활동
- ❖ Time Box: 10분
- ❖ 수행 방법
 - ✓ 점 스티커를 팀원들에게 나누어 줍니다.
 - ✓ 팀원들에게 마음 속으로 모아진 데이터에 대해 고민하도록 하고, 팀에 가장 도움이 되었던 것과 가장 방해가 되었던 것을 우선순위화하도록 합니다.
 - ✓ 팀원들에게 우선순위가 높은 순으로 두 가지에 대해 모아진 데이터가 있는 곳에 직접 가서, 투표하도록 합니다.
 - ✓ 표가 많이 있는 도움이 된 내용과, 방해가 된 내용 2 가지 주제에 대해 리포팅 합니다.
 - ✓ 15분 간 팀원들에게 도움이 된 내용은 어떻게 더 도움이 될 지, 방해가 된 내용은 어떻게 개선할 지에 대해 토의를 합니다.
 - ✓ 결론을 각 그룹의 대표가 이야기하게 합니다.
- ❖ 수행 팁
 - ✓ 토의에 적절한 인원은 4 명 정도입니다. 6 인 이상이면 그룹으로 나누어 토의합니다.

2. Retrospective 절차

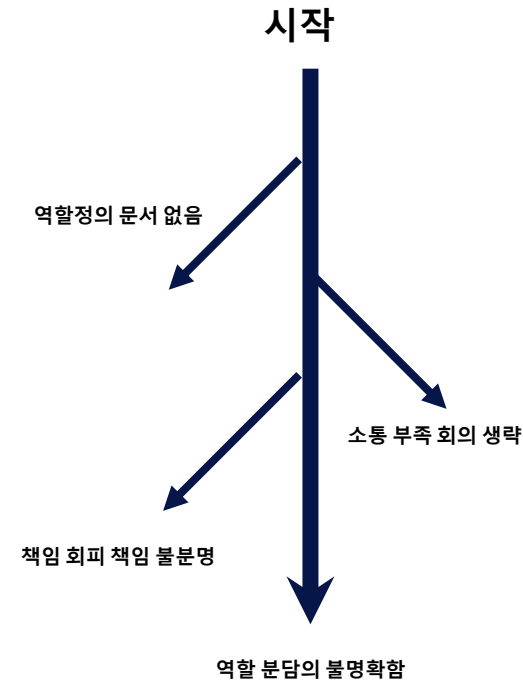


통찰 이끌어내기

Fishbone Diagram 예시



도움이 된 항목 예시: "명확한 목표 설정"



방해가 된 항목 예시: "역할 분담의 불명확함"



2. Retrospective 절차



무엇을 할 지 결정하기

SMART 한 회의

- ❖ Time Box: 20분
- ❖ 수행 방법
 - ✓ SMART 한 목표에 대해 설명합니다.
 - S: Specific (상세하고)
 - M: Measurable (측정 가능하고)
 - A: Attainable (달성 가능하고)
 - R: Relative (팀의 개선과 관련이 있으며)
 - T: Timely (이터레이션 안에 실행 가능해야 합니다)
 - ✓ 팀에게 15분의 시간을 주고, 주어진 데이터에 대해, 다음 이터레이션 동안 무엇을 SMART 하게 수행하면, 팀에 도움이 될 지 토의하도록 합니다.
 - ✓ 그룹 별 회의 결과를 그룹에서 한 명씩 타 팀과 공유할 수 있도록 리포팅합니다.
- ❖ 수행 팁
 - ✓ 인원이 4 인 이상이면 그룹으로 나누어 토의합니다.

※ KPT 회고 (Keep, Problem, Try)를 활용할 수 있습니다.



2. Retrospective 절차



무엇을 할 지 결정하기

SMART 한 회의

KPT 회고 (Keep, Problem, Try)

- Keep (계속 유지할 것들)
 - 무엇이 잘 되었는가?
 - 어떤 관행을 계속 유지해야 하는가?
 - 팀에게 도움이 된 것들은?
- Problem (문제점들)
 - 어떤 장애물이 있었는가?
 - 무엇이 팀을 느리게 했는가?
 - 어떤 프로세스가 비효율적이었는가?
- Try (시도해볼 것들)
 - 다음에는 무엇을 다르게 해볼까?
 - 어떤 새로운 도구나 방법을 시도해볼까?
 - 문제 해결을 위한 실험들은?

2. Retrospective 절차



무엇을 할 지 결정하기

EXACT한 목표기준 세우기

- ❖ Time Box: 20분
- ❖ 수행 방법
 - ✓ EXACT 한 목표에 대해 설명합니다.
 - EX: Exciting (긍정적이고 에너지가 넘치는 영감을 주고)
 - A: Assessable (성취한 것을 확인 가능하며)
 - C: Challenging (도전적인 목표이고)
 - T: Timely (기한이 있어야 합니다)
 - ✓ 팀에게 15분의 시간을 주고, 주어진 데이터에 대해, 다음 이터레이션 동안 무엇을 SMART 하게 수행하면, 팀에 도움이 될 지 토의하도록 합니다.
 - ✓ 그룹 별 회의 결과를 그룹에서 한 명씩 타 팀과 공유할 수 있도록 리포팅합니다.
- ❖ 수행 팁
 - ✓ 인원이 4 인 이상이면 그룹으로 나누어 토의합니다.



2. Retrospective 절차



회고 마무리하기

회고활동에 대한 회고 (+/델타(Delta))

- ❖ 수행한 회고를 마무리하고, 다음 이터레이션의 시작을 알리는 활동
- ❖ Time Box: 10분
- ❖ 수행 방법
 - ✓ 플립 차트에 T 자 형태의 표를 그리고 제한 시간을 알려줍니다.
 - ✓ 강점과 변화를 마음껏 말하도록 유도합니다.
 - ✓ 사람들의 피드백에 감사하며 더 발전시킬 것과 변화시킬 것을 정합니다.
- ❖ 수행 팁
 - ✓ T 자 형태의 표를 기준으로 피드백의 수를 제한하는 방법도 있습니다.
 - ✓ 변화시킬 것이 결정되었다면 담당자를 정해 추적하게 하는 것이 좋습니다.
- ❖ 기대효과
 - ✓ 회고를 끝내기 전에 다음 회고에서 이번 회고의 무엇을 유지하며 어떤 변화를 시도할지 결정할 수 있습니다.

회고 프로세스 개선	
+	△
잘된 점, 유지할 점을 의미합니다.	개선할 점 또는 변화가 필요한 부분을 의미합니다.



2. Retrospective 절차

회고 마무리하기



Follower 구하기

❖ Time Box: 10분

❖ 수행 방법

- ✓ 회고를 통해 결정한 사안의 중요성과 가치에 대해서 공유합니다.
- ✓ 회고를 통해 결정한 사안에 대해 추적 관리할 담당자를 구합니다.
- ✓ 자발적으로 문제를 추적 관리할 담당자가 결정되면 진행자는 일의 중요성을 다시 강조하며 모두 함께 담당자를 격려합니다.

❖ 수행 팁

- ✓ 강요 되지 않도록 주의합니다.

❖ 기대효과

- ✓ 팀원 사이에 자발적인 스폰서십을 구하여 자기조직적인 팀 분위기를 형성합니다.



3. 회고 진행 순서 템플릿

Uni

1

확인 설명-5분

❖ 사전 준비 단계

- ✓ 체크인
- ✓ 집중할 것/집중하지 말 것
- ✓ ESVP
- ✓ 작업규칙
- ✓ 체온측정
- ✓ 만족도 막대그래프

❖ 자료 모으기 단계

- ✓ 시간축
- ✓ 점 스티커로 색 표시하기
- ✓ 강점 알아내기

❖ 통찰 이끌어내기 단계

- ✓ 브레인 스토밍
- ✓ 필터링
- ✓ 생선가시
- ✓ 점 투표로 우선순위 매기기
- ✓ 학습 매트릭스



3. 회고 진행 순서 템플릿

Uni

1

확인 설명-5분

- ❖ 무엇을 할 지 결정하기 단계
 - ✓ 회고 계획 게임
 - ✓ SMART 목표
 - ✓ 순환 질문
- ❖ 회고 마무리하기 단계
 - ✓ +/-델타(Delta)
 - ✓ 체온 측정
 - ✓ 감사 표현하기
 - ✓ 시간 투자 대비 보상(ROTI)
 - ✓ 만족도 막대 그래프
 - ✓ 학습 매트릭스



3. 회고 진행 순서 템플릿

Uni

1

준비-5분

- ❖ 회고 주제 정하기
 - ✓ 년초 계획에 대해 회고
- ❖ 회고 규칙 정하기
 - ✓ 긍정적인 것
 - ✓ 개선점
- ❖ 회고준비
 - ✓ 자리 배치
 - ✓ 필요한 도구 챙기기



회고3. 회고 진행 순서 템플릿 진행

Uni

2

35분

- ❖ 사전 준비 단계
 - ✓ 5분
 - ✓ 활동
 - 체크인
- ❖ 자료 모으기 단계
 - ✓ 5분
 - ✓ 활동
 - 만족도 막대그래프
- ❖ 통찰 이끌어내기 단계
 - ✓ 10분
 - ✓ 활동
 - 학습 매트릭스
- ❖ 무엇을 할 지 결정하기 단계
 - ✓ 10분
 - ✓ 활동
 - 점 투표로 우선순위 매기기
- ❖ 회고 마치고 단계
 - ✓ 5분
 - ✓ 활동
 - -+/델타



3. 회고 진행 순서 템플릿



사전 준비 단계 : 5분

- ❖ 활동 - 체크인
 - ✓ 회고를 통해 기대하는 바 작성



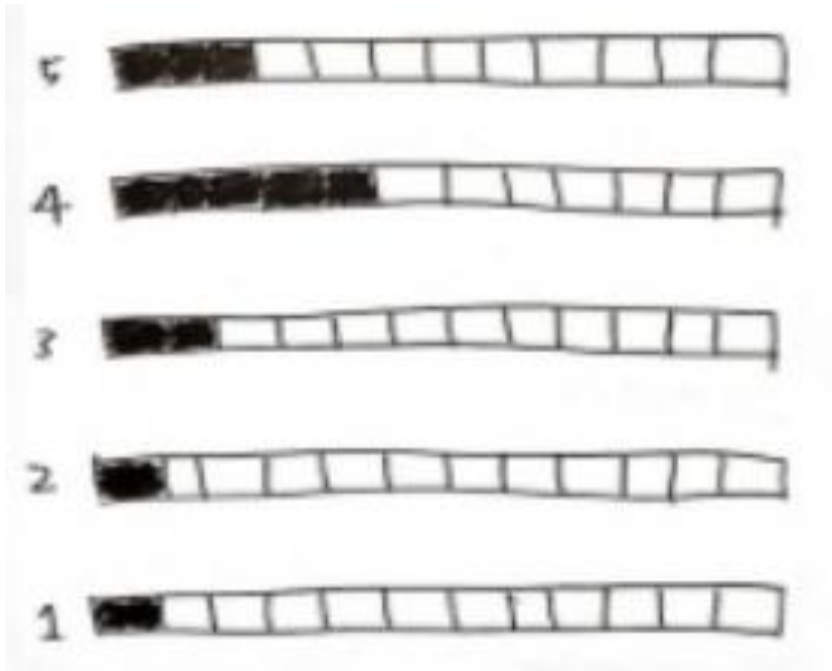
3. 회고 진행 순서 템플릿

Uni

2

자료 모으기 단계 : 5분

- ❖ 활동 - 만족도 막대그래프
 - ✓ 진행 한 내용에 대해 만족도 그래프 그리기





3. 회고 진행 순서 템플릿

Uni

2

통찰 이끌어내기 단계 : 10분

❖ 활동 - 학습 매트릭스

- ✓ '잘 했던 것', '다른 식으로 하고 싶은 것', '새로운 아이디어', '감사한 것' 순으로 자유롭게 작성

+	-
C	?



3. 회고 진행 순서 템플릿

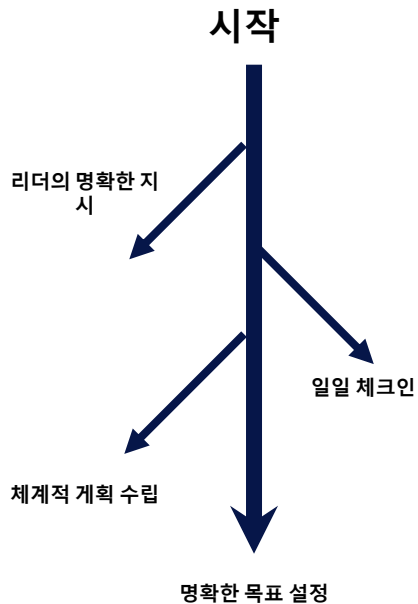
Uni

2

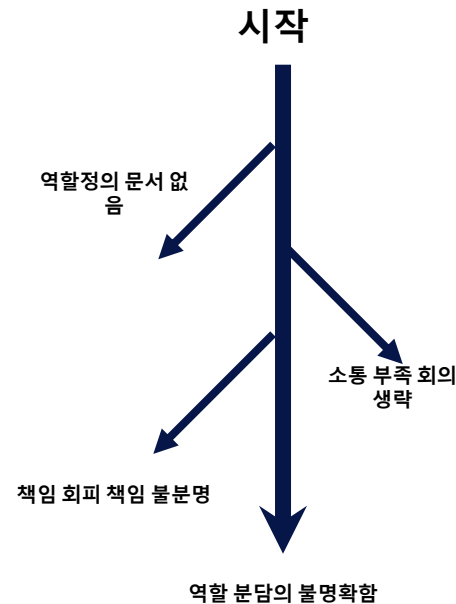
무엇을 할 지 결정하기 단계 : 10분

❖ 활동 - 점 투표로 우선순위 매기기

✓ 숫자가 많은 항목에 대해서 타당성에 문제가 없는지 검토



도움이 된 항목 예시: "명확한 목표 설정"



방해가 된 항목 예시: "역할 분담의 불명확함"



3. 회고 진행 순서 템플릿

Uni

2

회고 마치기 단계 : 5분

❖ 활동 - -+/델타

✓ 각자 적용할 만한 강점과 시도 할만한 변화에 대해서 정리

회고 프로세스 개선	
+	△

『4과목』

코딩 도구 기초 활용

- Part 1. 오리엔테이션
- Part 2. Cursor.AI 주요 기능 및 단축키
- Part 3. 토이 프로젝트 ① — Cursor 실습
- Part 4. Cursor + LangChain 통합
- Part 5. GitHub 연동 및 버전 관리
- Part 6. Prompting BP
- Part 7. 마무리 및 요약





1. Toy 프로젝트 실습(6시간)

코딩 도구 기초 활용

『4-1』 Project FluxNote - 초기 단계 (Initial Stage)

『4-2』 Project FluxNote - 고도화 단계 (Advanced Stage)

『4-3』 MCP 활용 회원가입 로그인 기반 게시판 만들기-Agora
커뮤니티 게시판

『4-4』 LangChain과 회사 챗봇 Prompt Project
실습 : Prompting BP



2. 핵심 요약

...

● ...

주제	핵심 정리
Cursor.AI	AI 코딩 IDE - 코드 생성 및 리팩토링 지원
LangChain	프롬프트 체인 및 도구 조합 프레임워크
GitHub	협업과 버전 관리의 중심
AI Workflow	반복 업무 자동화, 실시간 개선 가능 구조

THANK YOU.

앞으로의 엔지니어는 단순한 '코더'나 '기계 조작자'가 아니라 뇌-기계 인터페이스를 통해 지식과 능력을 즉각 확장하는 존재(뉴로-인터페이스: Neuro Interface)가 될 수 있습니다.

- 🎯 목표 달성을 위한 여정이 시작됩니다.
- 🌟 궁금한 점이 있으시면 언제든지 문의해주세요!
- 🚀 함께 코더와 프롬프트 전문가로 성장해 나갑시다!